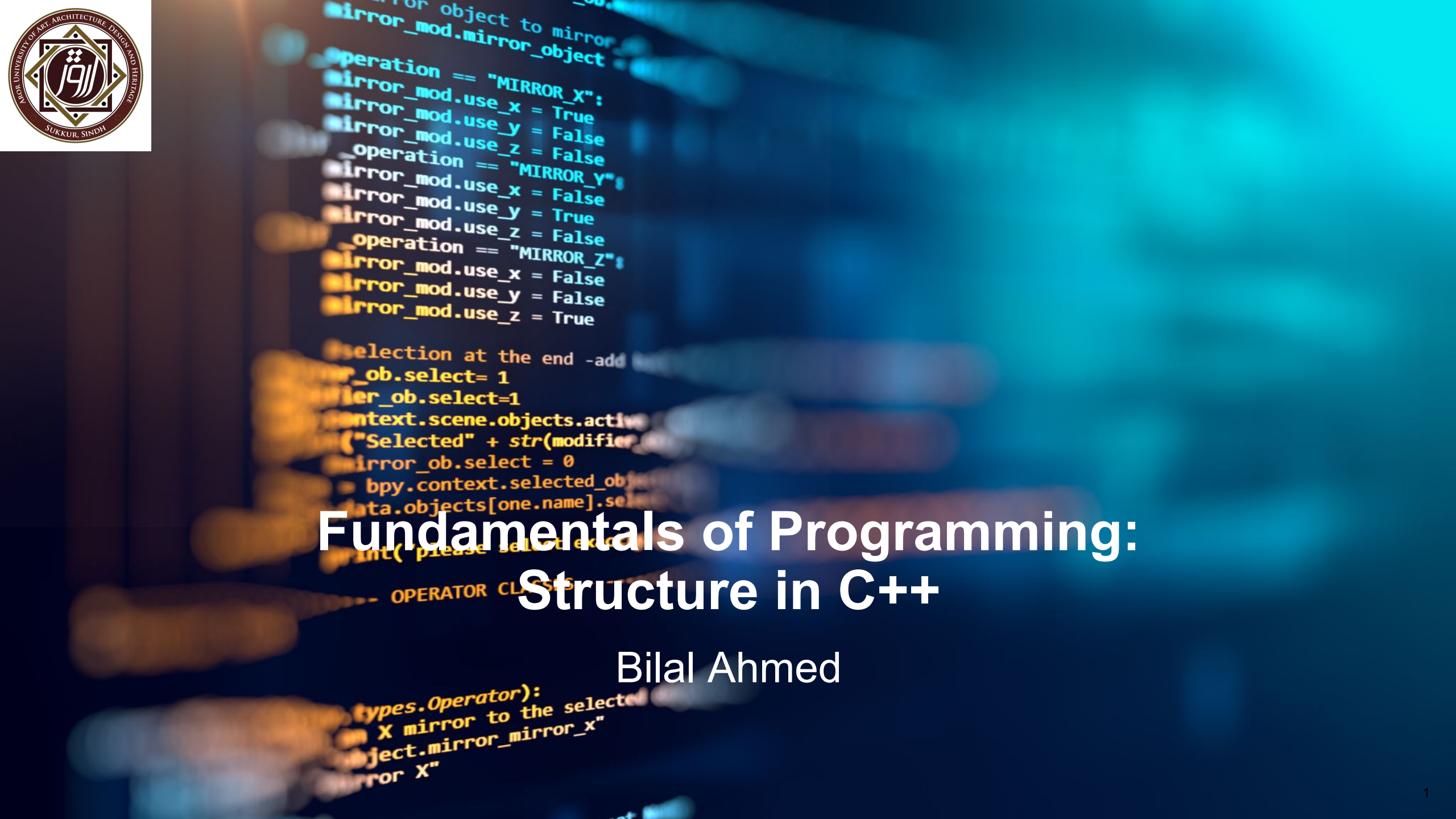


Fundamentals of Programming: Structure in C++

Bilal Ahmed

Agenda

- Need of Structure
- How to Create a Structure in C++
- Members of a Structure
- Dot . Operator
- Arrays inside Structure
- Functions inside Structure
- Array vs Structure
- Unnamed Structure

Concept of Structure

Problem: You want to store some information about a person: his/her name, citizenship number and gender:

Solution: Create different variables name, citNo, gender.

If, in the future, you would want to store information about multiple persons, what you will do ?

Concept of structure

you'd need to create different variables for each information per person: name1, citNo1, salary1, name2, citNo2, salary2

You can easily visualize how big and messy the code would look. Also, since no relation between the variables (information) would exist, it's going to be a scary task.

Concept of structure

A better approach will be to have a collection of all related information under a single name Person, and use it for every person.

Now, the code will look much cleaner, readable and efficient as well.

This collection of all related information under a single name Person is a structure.

What is a structure?

Collection of variables of different data types under a single name.

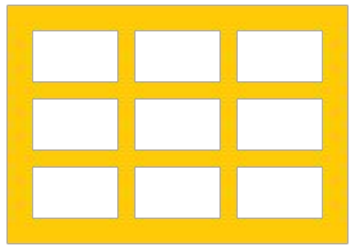
It is a **user defined data type** which groups together items of possibly **different data types** under a **single type**.

How to create a structure in C++

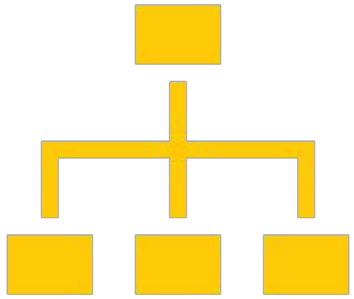
- struct keyword is used to create a structure in C++

```
struct structureName{
    member1;
    member2;
    member3;
    .
    .
    .
    memberN;
};
```

Types of structure members

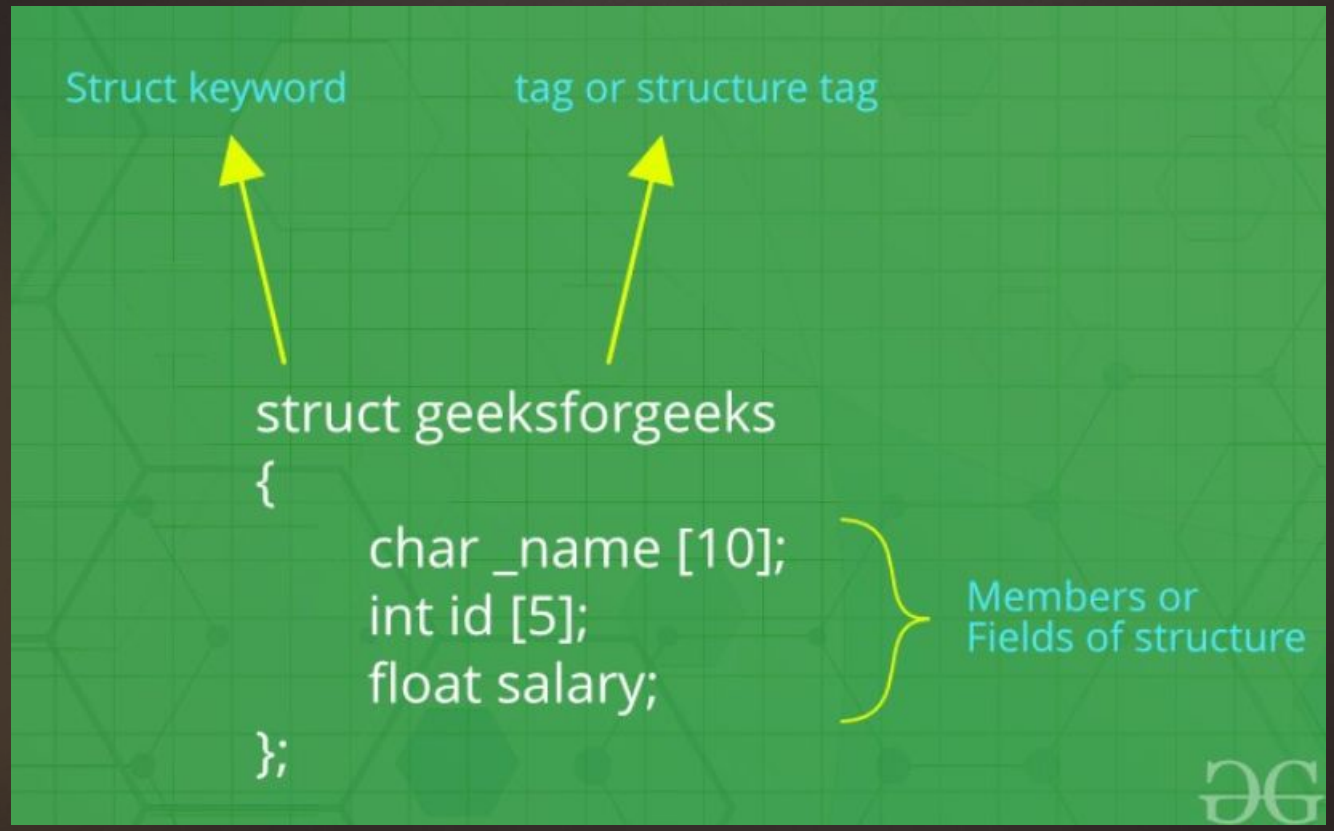


Data Members: Normal variables/Arrays in a structure



Member Functions: Normal C++ functions defined inside a structure

Example struct



```
//Creating a structure named person, with data members called age,name and gender
struct Person{
    float age;
    string name;
    char gender;
};
```

Declaration of a struct and its variables

Creating a structure variable


```
int main(){

//Creates two variables, p1 and p2 of Person data Type
Person p1;
Person p2;



return 0;
}
```

Dot (.) operator/ Member access operator

- It is used to access the members of structure:
  - For initializing/modification of structure member variables
  - For accessing the values of structure member variables

```
int main(){

Person p1;
p1.age=35;
p1.name="Ayan";
p1.gender='M';

return 0;
}
```

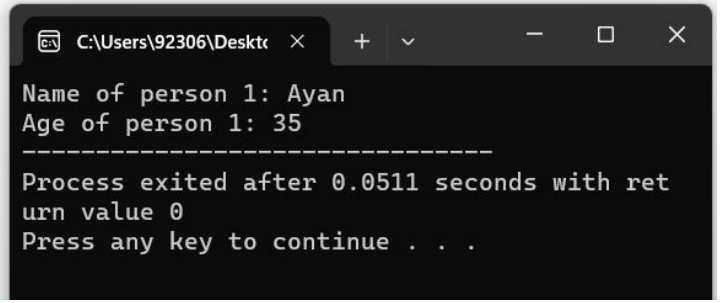
Initializing the variables of structure

```
int main(){

Person p1;
p1.age=35;
p1.name="Ayan";
p1.gender='M';

cout<<"Name of person 1: "<<p1.name<<endl;
cout<<"Age of person 1: "<<p1.age;

return 0;
}
```



Accessing the variables of structure

Example

```
#include <iostream>
using namespace std;

// Define a structure
struct Student {
    string name;
    int age;
    float marks;
};

int main() {
    // Create a structure variable
    Student s1;

    // Assign values
    s1.name = "Ali";
    s1.age = 20;
    s1.marks = 85.5;

    // Display values
    cout << "Name: " << s1.name << endl;
    cout << "Age: " << s1.age << endl;
    cout << "Marks: " << s1.marks << endl;

    return 0;
}
```

Task

- Create a structure called employee, an employee has name, emp_id and emp_salary.

- Inside main function assign structure members a value and print them using Dot Operator.

Using an array in struct

```
#include <iostream>
using namespace std;

// Structure with an array
struct Student {
    string name;
    int marks[3];   // array inside structure
};

int main() {
    Student s;

    s.name = "Ali";
    s.marks[0] = 85;
    s.marks[1] = 90;
    s.marks[2] = 95;

    cout << "Name: " << s.name << endl;
    cout << "Marks: "
         << s.marks[0] << ", "
         << s.marks[1] << ", "
         << s.marks[2] << endl;

    return 0;
}
```

Example

```
#include <iostream>
using namespace std;

// Structure with an array
struct Student {
    string name;
    string subjects[3];
    int marks[3];   // array inside structure
};

int main() {
    Student s;

    s.name = "Ali";
    s.marks[0] = 85; s.subjects[0]="English";
    s.marks[1] = 90; s.subjects[1]="khabar nahe";
    s.marks[2] = 95; s.subjects[2]="Tabahi";

    cout << "Name: " << s.name << endl;
    cout << "Marks: "
         << s.subjects[0] <<" : " <<s.marks[0] << ", "
         << s.subjects[1] <<" : " << s.marks[1] << ", "
         << s.subjects[2] <<" : " <<s.marks[2] << endl;

    return 0;
}
```

Using an array in struct

```
#include <iostream>
using namespace std;

// Structure with an array
struct Student {
    string name;
    int marks[3];   // array inside structure
};

int main() {
    Student s;

    s.name = "Ali";
    s.marks[0] = 85;
    s.marks[1] = 90;
    s.marks[2] = 95;

    cout << "Name: " << s.name << endl;

    cout << "Marks: ";
    for (int i = 0; i < 3; i++) {
        cout << s.marks[i] << " ";
    }

    return 0;
}
```

```
struct Dept{
    string dept_name;
    string emp_names[20];
    int building_number;
};
```

Using Array in a structure

```
Dept d1;
d1.name="Computer Science";
d1.building_number=4;
d1.emp_names[0]="Ahmed"


for(int i=1;i<20;i++){
    cin>>d1.emp_names[i];
}
```

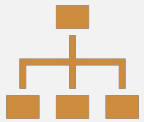
Using Array in a structure

Memory allocation in structure

- Once you declare a structure person as above. You can define a structure variable as:
- *Person ahmed;*
- Here, a structure variable **ahmed** is defined which is of type structure Person.
- When structure variable is defined, only then the required memory is allocated by the compiler.
- Memory of int is 4 bytes and memory of string is 8 byte. Hence, 12 bytes of memory is allocated for structure variable ahmed.

Scope of structure

Defined in global: So far, we have used global scope, which can be used anywhere, like in other functions as well.



Defined in local: If we define a structure inside main function, then we can only use it inside the main function, not outside the main function

Using a function in struct

```
#include <iostream>
using namespace std;

struct Student {
    string name;
    int age;

    // Function inside structure
    void display() {
        cout << "Name: " << name << endl;
        cout << "Age: " << age << endl;
    }
};

int main() {
    Student s;

    s.name = "Ali";
    s.age = 20;

    s.display();   // calling the function inside struct

    return 0;
}
```

Array of Structures

```
#include <iostream>
using namespace std;

struct Student {
    string name;
    int age;
};

int main() {
    // Array of 3 Student structures
    Student students[3];

    // Assign values
    students[0].name = "Ali";
    students[0].age = 20;

    students[1].name = "Sara";
    students[1].age = 19;

    students[2].name = "Ahmed";
    students[2].age = 21;

    // Display values
    for (int i = 0; i < 3; i++) {
        cout << students[i].name << " - " << students[i].age << endl;
    }

    return 0;
}
```

Array Vs Structure

Array

- Collection of homogeneous data.
- Data is accessed using Index.
- Allocates static memory.
- Access takes less time than structure.

Structure

- Collection of heterogeneous data.
- Data is accessed using dot (.).
- Allocates dynamic memory.
- Access takes more time than array.